Named Functions Are Always Global

As mentioned earlier, function identifiers are not subject to scoping rules. Here is a quick demonstration, notice that even though `inner()` is declared inside a local scope (that of the `outer()` function), it is still available globally.

```
outer();
inner();

function outer()
{
    function inner() {
        print "I am also global!". PHP_EOL;
    }
}

Output:
I am also global!
```

It is helpful to think of scope in terms of boundaries. The boundaries of local scope exist at the very beginning and end of the function block. All code that exists between the brace characters { } of the expression `function foo() { }` is locally scoped: the braces are the demarcations of the boundary. When the boundaries are crossed, scope changes. But in the case of function identifiers, as above, these are always put into global scope even if declared within another function's local scope. Only variables are scoped in PHP.

Some languages treat *all* blocks as having local scope – JavaScript generates a new local scope on every `if` block, for example – but this is not the case in PHP.

Function Arguments Are Always Local

The scope of variables defined as function parameters is always local:

```
$quote = "We've gone on holiday by mistake." . PHP_EOL;

function printQuote (string $quote): void {
    $quote = "We're bona fide, we're not from London" . PHP_EOL;
    print $quote;
}
```